Secure Microservices POC — Interview Reference

ASP.NET Core 8 · YARP · JWT · MassTransit · RabbitMQ · Entity Framework Core · SQL Server

.NET 8	Microservices	YARP Gateway	JWT Auth	RabbitMQ	MassTransit	EF Core	SQL Server	ASP.NET MVC

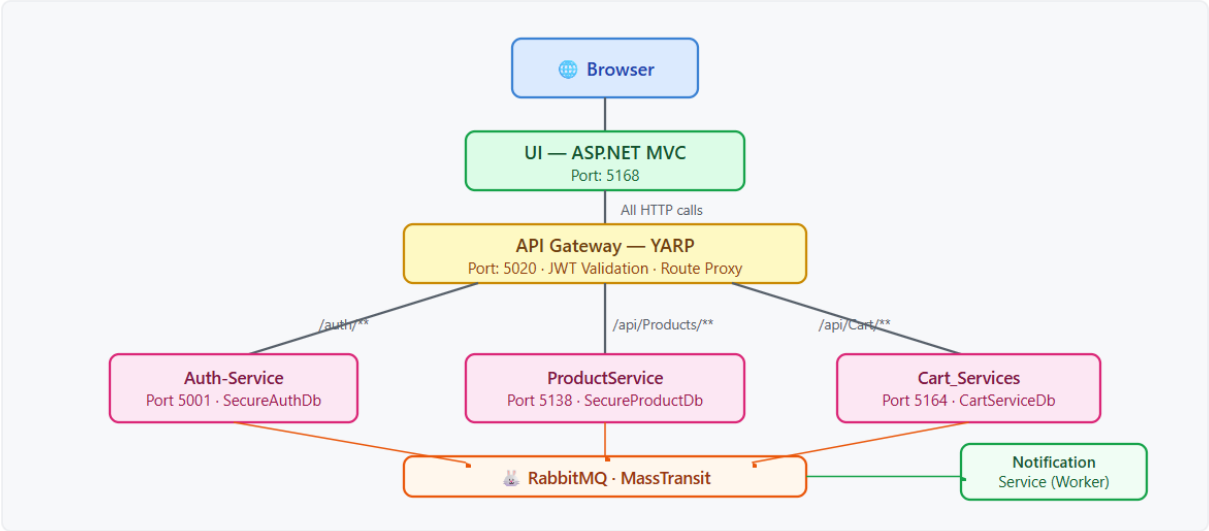
1 · Architecture Overview

Secure Microservices POC — Interview Reference

ASP.NET Core 8 · YARP · JWT · MassTransit · RabbitMQ · Entity Framework Core · SQL Server

.NET 8 | Microservices | YARP Gateway | JWT Auth | RabbitMQ | MassTransit | EF Core | SQL Server | ASP.NET MVC

1 · ARCHITECTURE OVERVIEW



Golden Rule (say this in interview): The UI never calls a microservice directly. Every single HTTP request goes through the API Gateway. The Gateway validates the JWT, then proxies to the correct downstream service using YARP route rules.

2 · SERVICES AT A GLANCE

Golden Rule (say this in interview): The UI never calls a microservice directly. Every HTTP request goes through the API Gateway. The Gateway validates the JWT, then proxies to the correct downstream service using YARP route rules.

2 · Services at a Glance

Service	**Port**	**Type**	**Database**	**Publishes Event**
UI	5168	ASP.NET Core MVC	—	—
API Gateway	5020	YARP Reverse Proxy	—	—
Auth-Service	5001	Web API	SecureAuthDb	UserRegisteredEvent
ProductService	5138	Web API	SecureProductDb	ProductChangedEvent
Cart_Services	5164	Web API	CartServiceDb	OrderPlacedEvent
NotificationService	—	.NET Worker	—	Consumes all 3 events
Shared	—	Class Library	—	Defines event contracts

3 · API Gateway (YARP)

Route	**Path Pattern**	**Upstream**
auth-route	/auth/{**catch-all}	:5001
products-route	/api/Products/{**catch-all}	:5138
cart-route	/api/Cart/{**catch-all}	:5164

Security Middleware

- Validates JWT on all requests (HS256)
- Blocks POST / PUT / DELETE to */api/products/** if not authenticated
- GET requests are open (no token needed to browse products)
- Global 500 error handler returns JSON

4 · Auth-Service

POST /auth/auth/register

- Validate username + password not empty
- Check username not already taken (DB)
- Save AppUser to SecureAuthDb
- Publish **UserRegisteredEvent** → RabbitMQ
- Return `{ message, username }`

POST /auth/auth/login

• Find user by username

• Compare password (plain text — demo only!)

• Build Claims: NameIdentifier, Name, fullName, Role

• Generate HS256 JWT (8-hour expiry)

• Return `{ access_token, username, fullName, role }`

AppUser Entity Field	**Type**	**Notes**
Id	int (PK)	Auto-increment
Username	string	Must be unique
Password	string	Plain text — demo only (use BCrypt in prod)
FullName	string	
Role	string	Default: "User"
CreatedUtc	DateTime	UTC timestamp

5 · ProductService

Verb	**Endpoint**	**Auth Required?**	**Notes**
GET	/api/Products	No	Supports ?category= and ?search= filters
GET	/api/Products/{id:guid}	No	Single product by GUID
POST	/api/Products	Yes (JWT)	Creates product; publishes ProductChangedEvent ("created")
PUT	/api/Products/{id:guid}	Yes (JWT)	Partial update — only supplied fields change
DELETE	/api/Products/{id:guid}	Yes (JWT)	Publishes ProductChangedEvent ("deleted")
POST	/api/Products/seed	No	Idempotent — seeds 12 demo products, skips existing SKUs

Product entity: Id(Guid), Sku(unique), Name(indexed), Brand, Category(indexed), Price, OriginalPrice, Rating(double), ReviewCount, ImageUrl, Badge?, IsNew, CreatedUtc

6 · Cart_Services

Verb	**Endpoint**	**Action**
GET	/api/Cart/{sessionId}	Get or create cart
POST	/api/Cart/{sessionId}/items	Add item (increments qty if exists)
PATCH	/api/Cart/{sessionId}/items	Delta +1/–1; auto-removes at qty ≤ 0
DELETE	/api/Cart/{sessionId}/items/{productId}	Remove one line
DELETE	/api/Cart/{sessionId}	Clear cart = Place Order

Database Schema

- **Carts:** Id, SessionId (unique), CreatedUtc, UpdatedUtc
- **CartItems:** Id, CartId (FK→Carts, cascade delete), ProductId, Name, Brand, Price, OriginalPrice, ImageUrl, Quantity

CartDto (returned on every response)

- SessionId, Items[], SubTotal
- TotalDiscount, GrandTotal, TotalItems
- Per-item: LineTotal, LineSaving

Session Identity: The ASP.NET Core session cookie ID is used as the cart key. No login is needed to add items to a cart — each browser session automatically has its own isolated cart.

7 · Async Messaging (RabbitMQ + MassTransit)

Event	**Published by**	**Consumed by**	**Fields**
UserRegisteredEvent	Auth-Service (on Register)	NotificationService	Username, FullName, RegisteredAtUtc
ProductChangedEvent	ProductService (Create/Delete)	NotificationService	ProductId, ProductName, Brand, Category, Price, ChangeType, ChangedAtUtc
OrderPlacedEvent	Cart_Services (ClearCart)	NotificationService	SessionId, GrandTotal, TotalItems, PlacedAtUtc

Pattern: All events are C# record types in the *Shared* class library — referenced by both publisher and consumer. MassTransit auto-creates queues on startup via cfg.ConfigureEndpoints(context).

8 · UI (ASP.NET Core MVC)

ShopController Actions

- **GET /Shop** — Products + cart count; supports ?category, ?search, ?sort
- **POST /Shop/AddToCart** — Requires login; fetches product, posts to cart
- **POST /Shop/BuyNow** — Add + redirect to cart
- **GET /Shop/Details/{id}** — Product JSON for modal
- **GET /Shop/Cart** — Full cart with order summary
- **POST /Shop/UpdateQuantity** — +1 / –1 on a line
- **POST /Shop/RemoveItem** — Remove full line
- **POST /Shop/PlaceOrder** — Clears cart, shows success banner

Typed HttpClient Services (all via Gateway)

- **ProductCatalogueService** → /api/Products/*
- **CartService** → /api/Cart/*
- **AuthService** → /auth/auth/login, /register
- **CommentService** → /api/products/{id}/comments

All clients share BaseAddress = API Gateway URL (configured in appsettings.json via ApiGateway:BaseUrl).

9 · End-to-End User Journey (9 Steps)

#	**User Action**	**Flow**
1	Open browser	Browser → UI:5168/Shop → ShopController.Index()
2	Products load	UI → Gateway:5020/api/Products → ProductService:5138 → SQL Server (SecureProductDb)
3	Product grid renders	Gateway returns JSON → UI passes to Views/Shop/Index.cshtml
4	Click "+ Cart"	UI → GET /api/Products/{id} (product details) + POST /api/Cart/{sid}/items → Cart_Services:5164 → CartServiceDb
5	Cart count in header	Every page load calls GET /api/Cart/{sid} → returns TotalItems
6	View cart	UI → GET /api/Cart/{sid} → Cart_Services → full CartDto with totals
7	Update quantity	UI → PATCH /api/Cart/{sid}/items (delta +1/–1) → Cart_Services
8	Remove item	UI → DELETE /api/Cart/{sid}/items/{productId} → Cart_Services
9	Place order	UI → DELETE /api/Cart/{sid} → Cart_Services clears cart + publishes OrderPlacedEvent → RabbitMQ → NotificationService logs confirmation

10 · Startup Order & Setup

- Start SQL Server locally 2. Start RabbitMQ (default: localhost:5672, guest/guest) 3. dotnet run ProductService → port 5138 (auto-migrates SecureProductDb) 4. dotnet run Cart_Services → port 5164 (auto-migrates CartServiceDb) 5. dotnet run Auth-Service → port 5001 (auto-migrates SecureAuthDb) 6. dotnet run API-Gateway → port 5020 7. dotnet run NotificationService → Worker (no port) 8. dotnet run UI → port 5168 Seed demo products (run once): POST http\://localhost:5020/api/Products/seed

11 · Key Technology Decisions (Interview Talking Points)

Concern	**Choice**	**Why**
API Gateway	YARP (Yet Another Reverse Proxy)	Microsoft-native, config-driven, integrates directly with ASP.NET Core middleware pipeline — easy to add auth/logging at the edge
Authentication	Custom JWT (HS256) — no ASP.NET Identity	Lightweight, no overhead of Identity tables; full control over claims and token lifetime
Async messaging	MassTransit + RabbitMQ	Decouples services — Auth/Product/Cart don't need to know about Notification; events are fire-and-forget with guaranteed delivery via queue
ORM	EF Core 8 + SQL Server	Code-first migrations, LINQ queries, strongly-typed repositories
Cart identity	ASP.NET Core Session Cookie	No login required to shop; each browser session has its own isolated cart
Write protection	Inline middleware in API Gateway	Centralised — downstream services don't need their own auth logic for POC
Event contracts	Shared Class Library (C# records)	Single source of truth — publisher and consumer always agree on the event schema

12 · POC Limitations & Production Improvements

Passwords stored in plain text — Replace with BCrypt / Argon2 hashing in production.

JWT secret in appsettings.json — Move to environment variables or Azure Key Vault; rotate regularly.

No refresh tokens — Sessions expire after 8 hours; users must re-login. Add a refresh token endpoint.

NotificationService only logs — Wire to real email (SendGrid) / SMS (Twilio) in production.

No Docker / Kubernetes — Each service runs manually. Add Dockerfiles + docker-compose or Helm charts for real deployment.

No health checks or circuit breakers — Add ASP.NET Core Health Checks + Polly for resilience.

13 · Quick Interview Q&A

Q: Why use a gateway instead of direct calls?

- Single entry point — easy to add cross-cutting concerns (auth, logging, rate limiting)
- UI doesn't need to know the address of each microservice
- Downstream services can move ports without UI changes

Q: Why MassTransit instead of raw RabbitMQ?

- Abstracts queue/exchange setup — swap to Azure Service Bus with 1 line change
- Auto-creates queues (ConfigureEndpoints)
- Built-in retry policies, dead-letter handling

Q: How is the cart identified without login?

- ASP.NET Core session creates a unique cookie per browser
- That session ID is used as the CartId in CartServiceDb
- When the user logs in, the same session ID continues — cart persists

Q: How does JWT security work here?

- Auth-Service issues the token (HS256, 8h expiry)
- UI stores token in ASP.NET Session (HttpOnly cookie)
- UI sends Bearer token in Authorization header for write calls
- API Gateway validates token before forwarding

Q: How would you scale this in production?

- Containerise each service with Docker
- Orchestrate with Kubernetes — scale ProductService/CartService independently
- Use Redis for distributed session instead of in-memory
- Add API rate limiting and circuit breakers (Polly)

Q: What is the Shared library for?

- Contains only event record types (contracts)
- Both publisher and consumer reference the same assembly
- Guarantees schema consistency across services
- No business logic — pure DTOs